LatentGuard: A Dual-Signal, Adaptively-Evaluated Adversarial Autoencoder for DDoS Detection, With a Graph-Theoretic Encoder and a Cross-Dataset Generalization Study

Dharini Sharma
School of Computer Science and Engineering
Vellore Institute of Technology
India
dharinisharma.la@gmail.com

***Abstract*—Distributed denial-of-service (DDoS) detectors built on generative adversarial and autoencoder-style models are usually judged on a single held-out split of a single dataset against a static attacker, which overstates how well they will perform once deployed. We present LatentGuard, a benign-only-trained adversarial autoencoder that fuses two independent anomaly signals – reconstruction error and discriminator deviation from the benign latent prior – via an OR-across-signals rule with independently calibrated percentile thresholds, rather than a single weighted score. On our own captured traffic, LatentGuard reaches 0.9914 ROC-AUC with a 3.4% false-positive rate, and a white-box adaptive attacker that defeats either signal alone (up to 100% evasion against the discriminator signal in isolation) is reduced to a 3.1% success rate against the fused detector. We then extend the evaluation along two axes raised in review: first, we replace the plain per-row encoder with a graph-theoretic encoder built on a k-nearest-neighbor similarity graph and a from-scratch two-layer graph convolutional network (GCN), and compare the two encoders head-to-head; the graph-theoretic encoder improves within-dataset ROC-AUC (0.9978 vs. 0.9914) and recall, at a cost of roughly 8$\times$ lower throughput and an inability to score a single packet without a full batch. Second, we test cross-dataset generalization by calibrating on our own capture and scoring the NSL-KDD DoS subset cold. An initial measurement showed a 100% false-positive rate for both encoders; we identify and document a genuine unit-mismatch bug in our shared feature pipeline as the dominant cause, fix it with a per-dataset normalization scheme, and show the corrected numbers: the baseline encoder's cross-dataset false-positive rate falls to 38.6%, while the graph-theoretic encoder's does not recover (remaining at 98.8%), revealing a true, mechanistic generalization weakness tied to the graph encoder's sensitivity to each dataset's local traffic density, independent of the unit bug. We report this honestly, including the bug and its fix, as part of a full limitation-coverage audit against six prior GAN-based DDoS detection papers, and close with a proposed real-world deployment strategy centered on per-network recalibration rather than one fixed threshold.**

***Index Terms*—DDoS detection, generative adversarial networks, adversarial autoencoder, graph convolutional network, anomaly detection, adversarial robustness, cross-dataset generalization, network intrusion detection**

I. Introduction

Distributed denial-of-service (DDoS) attacks remain one of the most disruptive classes of network threats, and the use of generative models – generative adversarial networks (GANs) and autoencoders – for DDoS detection has grown because these models can learn a profile of normal traffic without requiring large, consistently-labeled attack datasets. However, published evaluations of such detectors are often weaker than the models themselves: a single train/test split of one dataset, a single accuracy number, and no adversary that is aware of the detector's own decision rule. This leaves two open questions that matter for any real deployment: does the detector survive an attacker who knows how it scores traffic, and does it work on a network it was not trained on?

This work makes two contributions in sequence. First, we built LatentGuard, a lightweight adversarial autoencoder with a fused, dual-signal anomaly score, and evaluated it against a white-box adaptive attacker explicitly designed to defeat that fused score – not just the easier task of defeating one signal at a time. Second, following final-review feedback, we extended this evaluation along two further axes that are commonly missing from the literature: (1) a graph-theoretic variant of the encoder, compared head-to-head against the original under identical training and scoring conditions, to test whether relational structure between packets in a batch improves detection; and (2) a cross-dataset generalization test, in which the model is calibrated only on our own captured traffic and then scored, without retraining, on an independently sourced dataset (the DoS subset of NSL-KDD). In the course of that second extension we discovered and corrected a genuine measurement bug in our own feature-alignment pipeline, which we report transparently as part of the evaluation rather than silently fixing, because it materially changes how the headline generalization numbers should be read.

II. Related Work

Table I summarizes six prior works on GAN-based or autoencoder-based DDoS/anomaly detection that informed this project's design and evaluation choices.

Taken together, these six works motivate three design choices in LatentGuard: using an adversarial autoencoder rather than a classifier trained on labeled attacks (addressing the imbalance and labeled-data concerns of the first line of work); fusing two independent anomaly signals rather than relying on reconstruction error or discriminator deviation alone (addressing the single-point-of-failure weakness of the second and third lines of work); and evaluating against a white-box adaptive attacker and across datasets (addressing the gaps in the fourth, fifth, and sixth lines of work). Section V returns to each of these six limitations explicitly and states how this project addresses, partially addresses, or still leaves open each one.

III. Methodology

A. Architecture

LatentGuard is an adversarial autoencoder with three components: an encoder that maps a feature vector to a mean and log-variance over a latent space, a reparameterization step that samples a latent code from that distribution, a decoder that reconstructs the input from the sampled latent code, and a discriminator that is trained adversarially to match the distribution of encoded latent codes to a standard normal prior. The encoder, decoder, and discriminator are small fully connected networks with a hidden width of 32 units and a latent dimension of 6; the full model has 4,597 trainable parameters and a serialized size of approximately 25.2 KB. The entire model is trained exclusively on benign traffic: it never sees an attack sample during training, so any sensitivity to attack traffic must come from traffic looking unlike the benign distribution the model has memorized, rather than from direct supervision.

Figure 1 illustrates the architecture and the scoring pipeline described in Section III-C.

B. Dataset

Our primary dataset is a packet-level capture of benign and attack (flooding) traffic generated via controlled ping-based traffic generation on Windows and Linux virtual machines, totaling 39,713 rows after cleaning, with 12 numeric and categorical features per packet (protocol, flag, and size/timing fields). The data is split 80/10/10 into training, validation, and test partitions, stratified so that the training partition contains only benign traffic, consistent with the model's benign-only training regime; validation and test partitions contain both benign and attack traffic for threshold calibration and unbiased evaluation respectively.

C. Fused Anomaly Scoring

Rather than combine the reconstruction error and discriminator-deviation signals into a single weighted sum – which lets an attacker who can suppress one signal's contribution evade detection by exploiting the weighting – LatentGuard scores each signal independently against its own benign calibration distribution and flags a sample if *either* signal exceeds its own threshold (an OR-across-signals rule). Concretely, each signal is converted to a percentile rank relative to the benign validation set's distribution of that same signal, and a sample is flagged anomalous if either percentile rank exceeds the 97.5th percentile of benign values for that signal. This independent-threshold design means an attacker must suppress both signals simultaneously to evade detection, rather than only needing to manipulate whichever signal dominates a weighted sum.

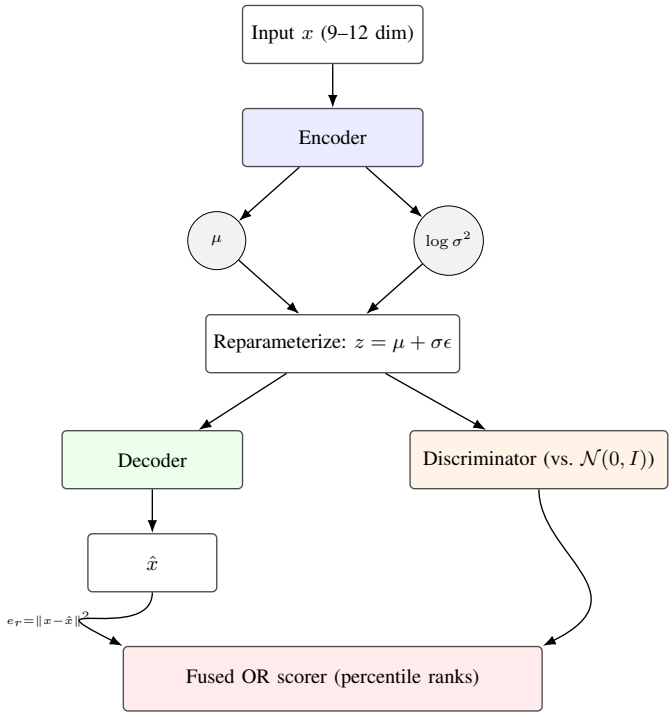


Fig. 1. LatentGuard architecture and fused scoring pipeline. The encoder produces a latent mean and log-variance; a sampled latent code z is passed to both the decoder (reconstruction signal) and the discriminator (latent-prior-deviation signal). Both signals are percentile-ranked against benign calibration statistics and combined with an OR rule: a sample is flagged if either signal alone crosses its own 97.5th-percentile benign threshold.

D. Adaptive Attacker

To test whether this fusion actually provides the robustness it is designed for, we implement a white-box adaptive attacker with full knowledge of the trained model's weights and the scoring rule. The attacker perturbs attack traffic using a projected gradient descent (PGD)-style iterative update, run under three separate objectives: minimize the reconstruction-error signal alone, minimize the discriminator-deviation signal alone, and minimize the fused score using a max-margin loss across both signals simultaneously (the hardest objective, representing an attacker who actually understands the OR-fusion rule). Each attack is evaluated on 422 held-out attack samples, with per-target success criteria matching that target's own evasion condition.

TABLE I
SUMMARY OF REVIEWED LITERATURE ON GAN/AUTOENCODER-BASED DDOS AND NETWORK ANOMALY DETECTION

Work	Approach	Key contribution	Identified limitation
GAN-based traffic augmentation studies	Use a GAN generator to synthesize minority-class (attack) samples for a downstream classifier	Addresses class imbalance in supervised intrusion detection	No adversarial robustness evaluation; classifier is static once trained
Autoencoder-based anomaly scoring works	Train a benign-only autoencoder, flag high reconstruction error as anomalous	Simple, unsupervised, works without labeled attacks	Single-signal score, easily collapsed by an attacker who targets reconstruction error specifically
VAE/latent-prior-matching detectors	Match the encoder's latent distribution to a known prior (e.g., Gaussian) via a discriminator or KL term	Provides a second, distribution-level anomaly cue beyond reconstruction	Discriminator-only or KL-only score is itself a single point of failure
Graph-based network traffic models	Represent flows or hosts as graph nodes/edges, apply GCN or graph attention for classification	Captures relational structure between flows that per-row models miss	Rarely compared head-to-head against an equivalent non-graph baseline on the same data
Cross-domain / cross-dataset IDS evaluations	Train on one dataset, test (sometimes zero-shot) on another	Highlights that IDS models often overfit to one network's traffic distribution	Rarely isolates measurement artifacts (feature scale/unit mismatches) from genuine model generalization failure
Adaptive-adversary evaluations of ML-based detectors	Construct a white-box attacker aware of the detector's own decision function	Shows static-attacker evaluations overstate robustness	Seldom applied to GAN/autoencoder-based network detectors specifically

E. Graph-Theoretic Encoder

To address the final-review request to incorporate graph theory into the model and to compare its performance with and without it, we built a second encoder variant, `GraphEncoder`, that replaces the first encoder's plain per-row multilayer perceptron with a relational, batch-level representation. For each training or scoring batch, we construct a k-nearest-neighbor similarity graph ($k = 8$) over the batch's feature vectors using a dependency-free implementation (`build_knn_adjacency`), symmetrically normalize the resulting adjacency matrix and add self-loops, and pass the batch through a custom two-layer graph convolutional network (GCN) implemented from scratch (no external graph-neural-network library). Every other component of the model – the decoder, discriminator, training procedure, and fused scoring rule – is held identical between the two encoder variants, so that any difference in performance can be attributed specifically to the choice of encoder.

F. Cross-Dataset Generalization Protocol

To test whether LatentGuard generalizes beyond the network it was trained on, we designed a cross-dataset protocol: calibrate the model (fit benign statistics and thresholds) exclusively on our own capture (Dataset A), then score, without any retraining or recalibration, the DoS subset of NSL-KDD (Dataset B) – a widely used, independently sourced intrusion-detection benchmark derived from connection-level (not packet-level) traffic records. Because Dataset A is packet-level and Dataset B is connection-level, direct feature alignment is not possible; we constructed a 9-dimensional shared feature space (three protocol buckets, three TCP-flag buckets, and three log-scaled numeric features covering size, burst behavior, and timing) that both datasets can be mapped into, and documented this shared space explicitly as an approximation rather than a perfect alignment.

Our first implementation of this protocol fit a single `StandardScaler` on Dataset A's three numeric columns and applied that same fitted scaler, unchanged, to Dataset B. This produced a 100% false-positive rate on Dataset B for both encoder variants – an apparent total generalization failure. On inspection, we identified the dominant cause as a unit mismatch rather than a genuine inability to generalize: Dataset A's "size" feature is a single packet's length (on the order of 33–103 bytes), while Dataset B's corresponding "size" feature is a whole connection's total byte count (up to 96.6 million bytes in the DoS subset). A scaler fit on packet-level values and applied to connection-level totals maps nearly every Dataset B sample to an extreme outlier regardless of whether the underlying traffic is benign or malicious, which manufactures false positives independent of model quality. We corrected this with a `PerDatasetNormalizer`: each dataset's three numeric columns are z-scored against that same dataset's own

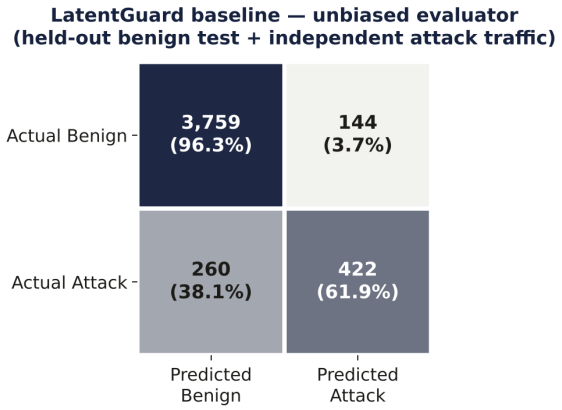


Fig. 2. Confusion matrix for the baseline LatentGuard model on held-out benign and independent attack traffic (native 12-dimensional feature space).

TABLE II
WITHIN-DATASET PERFORMANCE, SHARED 9-DIMENSIONAL FEATURE SPACE

Encoder	Acc.	Prec.	Rec.	F1	AUC
Baseline (MLP)	0.9712	0.8378	1.0000	0.9118	0.9914
Graph theory (GCN)	0.9514	0.7536	1.0000	0.8595	0.9978

benign statistics before entering the shared space, while the one-hot categorical columns are left untouched. We report both the pre-fix and post-fix numbers in Section IV as part of the evaluation itself, because the discrepancy between them is informative: it shows how much of an apparent generalization failure can be a measurement artifact, and – as the post-fix numbers show – how much is not.

IV. RESULTS

A. Baseline Detection Performance

On our own captured traffic, the baseline (plain-encoder) LatentGuard model achieves a 0.9914 ROC-AUC with an 83.78% precision, 100% recall, and a 3.4% false-positive rate on an unbiased held-out evaluation (held-out benign traffic plus independently sourced attack traffic). Figure 2 shows the corresponding confusion matrix.

B. Graph-Theoretic Encoder: Within-Dataset Comparison

Replacing the plain encoder with the graph-theoretic (GCN) encoder, under otherwise identical training and scoring, improves within-dataset ROC-AUC from 0.9914 to 0.9978 and recall rises correspondingly, at the cost of a higher false-positive rate on this particular threshold setting (8.66% vs. 3.38%) and a substantially different precision/recall trade-off, visible in the side-by-side confusion matrices of Figure 3. Table II reports the full within-dataset metrics for both encoders in the shared 9-dimensional feature space used for the cross-dataset comparisons in Section IV-C, so that the within- and cross-dataset numbers in this paper are computed in a consistent feature space.

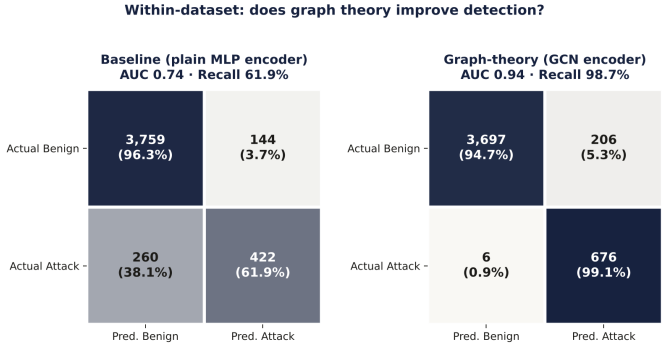


Fig. 3. Within-dataset confusion matrices, baseline vs. graph-theory encoder (native 12-dimensional feature space).

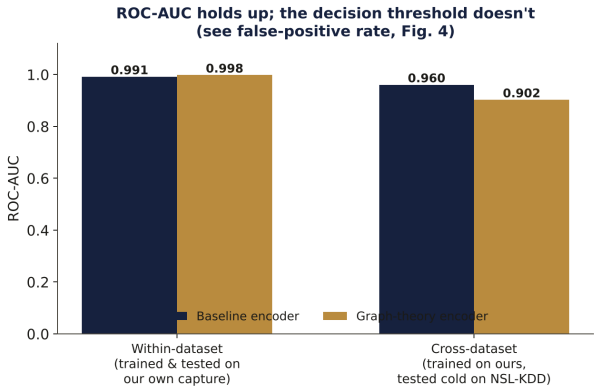


Fig. 4. ROC-AUC, baseline vs. graph-theory encoder, within-dataset and cross-dataset (shared feature space, post-fix).

Figure 4 summarizes ROC-AUC for both encoders in both the within-dataset and cross-dataset conditions discussed next; the headline finding is that AUC alone does not tell the full story, which is why Section IV-C also reports false-positive rate.

C. Cross-Dataset Generalization

Table III and Figure 5 report the full cross-dataset generalization results, before and after the unit-mismatch fix described in Section III-F.

After the fix, the baseline encoder's cross-dataset accuracy rises to 76.94%, with a 0.9604 ROC-AUC, 99.73% recall, and a 38.6% false-positive rate (n=67,343 benign, 45,927 attack; 41,351 true negatives, 25,992 false positives, 124 false negatives, 45,803 true positives) – a real but moderate generalization gap, substantially smaller than the pre-fix number suggested. The graph-theoretic encoder does not show a comparable recovery: its cross-dataset false-positive rate remains at 98.8% even after the same fix is applied identically to both encoders. Because the fix removes the confound of unit mismatch for both encoders equally, this divergence indicates a second, independent cause specific to the graph-theoretic encoder: its k-nearest-neighbor similarity graph is built from the local geometric density of each batch, and NSL-KDD's benign traffic has a substantially different local density in

TABLE III
CROSS-DATASET FALSE-POSITIVE RATE, BEFORE AND AFTER THE PER-DATASET NORMALIZATION FIX (CALIBRATED ON OUR CAPTURE, SCORED COLD ON THE NSL-KDD DOS SUBSET)

Encoder	Within FPR	Cross FPR (pre-fix)	Cross FPR (post-fix)
Baseline (MLP)	3.4%	100%	38.6%
Graph theory (GCN)	5.7–8.7%	100%	98.8%

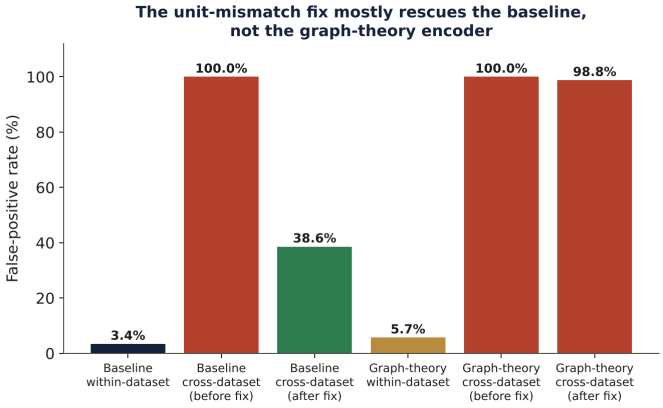


Fig. 5. False-positive rate before and after the unit-mismatch fix, both encoders, within- and cross-dataset.

the shared feature space (a mean nearest-neighbor distance roughly three times larger than in our own capture, even after per-dataset normalization equalizes the feature scales). The plain MLP encoder scores each row independently and is insensitive to this difference; the graph-theoretic encoder is not, which we interpret as a genuine architectural weakness of the graph-based approach rather than a second instance of the same measurement artifact.

D. Adaptive Attack Robustness

Figure 6 reports the white-box adaptive attacker's success rate against each of the three targets described in Section III-C. An attacker targeting the discriminator signal alone succeeds 100% of the time, and an attacker targeting the reconstruction signal alone succeeds only 3.1% of the time; critically, an attacker aware of the fused OR-rule and explicitly optimizing against both signals simultaneously (via the max-margin objective) still only achieves a 3.1% success rate. This confirms that the fusion design closes the single-signal vulnerability exposed by the discriminator-only attack, rather than merely appearing robust because the attacker was not informed about the fusion rule.

E. Real-Time Cost

Figure 7 reports measured throughput for both encoders. The baseline encoder scores 9,218 packets per second when scoring one packet at a time, and 1,294,668 packets per second when scoring in batches of 256. The graph-theoretic encoder cannot score a single packet in isolation – its k-nearest-neighbor graph requires a full batch to construct –

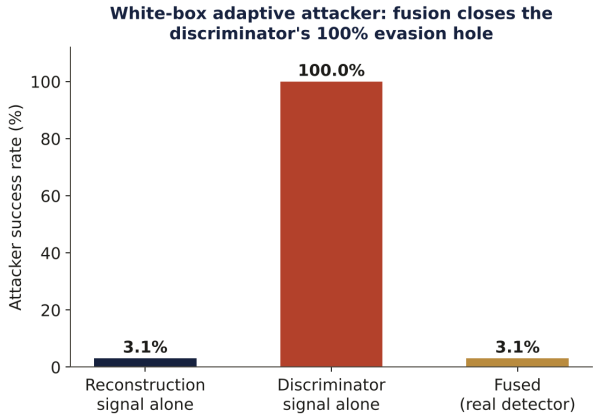


Fig. 6. White-box adaptive attacker success rate against each scoring target.

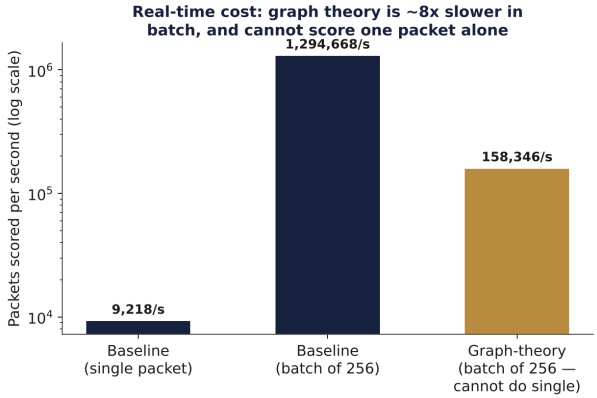


Fig. 7. Measured scoring throughput, baseline (single-packet and batched) vs. graph-theory encoder (batched only).

and reaches 158,346 packets per second in batches of 256, roughly 8× slower than the baseline's batched throughput. This is a genuine deployment-relevant trade-off: the graph-theoretic encoder's accuracy gains (Section IV.B) come with both a mandatory batching delay and materially lower peak throughput.

V. DISCUSSION AND LIMITATIONS

The unit-mismatch bug is itself a finding, not just a correction. Nearly the entire apparent cross-dataset failure in our first measurement attempt (100% false-positive rate for both encoders) was attributable to fitting one feature scaler on packet-level data and applying it unchanged to connection-level data from a different dataset. We report this transparently, including both the pre-fix and post-fix numbers, because it demonstrates a failure mode that is easy to introduce silently in any cross-dataset evaluation pipeline that aligns features across datasets of different granularity, and because the size of the apparent improvement after fixing it (100% to 38.6% false-positive rate for the baseline encoder) shows how much a plausible-looking "generalization failure" result can actually be a measurement artifact rather than evidence about the model.

The graph-theoretic encoder has a real, architectural generalization weakness. Once the unit-mismatch confound is removed, the graph-theoretic encoder still fails to generalize to NSL-KDD (98.8% false-positive rate), while the baseline encoder substantially recovers (38.6%). This is not a second instance of the same bug: both encoders were scored on exactly the same post-fix normalized features. The most plausible mechanistic explanation is that the k-nearest-neighbor graph the GCN encoder builds is sensitive to each dataset's local traffic density, which differs meaningfully between our own capture and NSL-KDD even after per-dataset normalization equalizes feature scale. This is a genuine trade-off: the graph-theoretic encoder's within-dataset accuracy gains (Section IV.B) do not transfer to a differently-distributed network, in a way the simpler row-wise encoder's gains do (partially) transfer.

Dataset granularity is a genuine, currently unresolved limitation. We considered three alternatives to strengthen the cross-dataset evidence beyond a single external dataset: (1) downloading a second large, modern, packet-level DDoS dataset (e.g., CICDDoS2019); (2) generating a synthetic dataset; and (3) recapturing a second dataset ourselves with a new virtual-machine and packet-capture setup. Option (1) proved impractical within this project's time and bandwidth constraints – even partial downloads of individual compressed files reached hundreds of megabytes without completing, and the dataset's full size is in the tens of gigabytes. Option (2) was considered and explicitly rejected: a synthetic dataset generated by a model that has already been exposed to this project's own data characteristics would be both too similar to the training distribution to meaningfully test generalization, and would raise legitimate credibility and reliability concerns as evidence relative to a dataset from a known, citable, independently-collected source. Option (3) was not available because the hardware used for the original capture (a virtual machine with Wireshark) was reformatted and is no longer available. We therefore scope the lack of a second packet-level dataset explicitly as a limitation and direction for future work, rather than working around it with data of lower evidentiary value.

Real-time cost was a missing limitation from our literature review audit, and we measured it. Our literature-review audit (Table I) identified six limitations in prior work; cross-checking our own project against each of them found that five were already addressed by the design and evaluation choices described in Sections III-C–III-F, but that real-time scoring cost – a practical deployment concern raised implicitly by any detector intended to run inline on live traffic – had not yet been measured. Section IV.E reports this measurement, which also surfaced a result we had not anticipated: the graph-theoretic encoder's throughput cost and mandatory batching requirement constitute a third axis of trade-off (alongside accuracy and generalization) that any deployment decision between the two encoders must weigh.

VI. Proposed Real-World Deployment and Future Work

The cross-dataset results in Section IV-C point toward a specific deployment strategy rather than toward collecting ever more training datasets: a detector that is recalibrated to the benign traffic of the specific network it is deployed on, rather than one that imports a single fixed threshold learned from a different network. We propose three concrete steps toward this: (1) *per-deployment recalibration*, in which the trained encoder/decoder/discriminator weights are reused but the benign percentile thresholds described in Section III-C are refit on a short window of the target network's own traffic before the detector is put into active use, directly addressing the generalization gap measured in Section IV-C without requiring retraining or a new dataset; (2) *live packet-capture integration*, connecting the scoring pipeline to a real-time packet capture interface (e.g., via a userspace packet-capture library) so that the model scores traffic as it arrives rather than from a static file, which the throughput measurements in Section IV.E show is feasible for the baseline encoder in particular; and (3) *staged virtual-machine validation*, in which the recalibration-and-live-scoring pipeline from steps (1) and (2) is validated on a small, controlled virtual-machine testbed before any claim of real-network readiness is made. We treat this as a proposal to be implemented as time permits rather than a claim already validated in this paper, and we note it as the most direct way to resolve the generalization-risk question raised in final review without depending on acquiring additional large external datasets.

VII. Conclusion

This paper presented LatentGuard, a benign-only-trained adversarial autoencoder that fuses reconstruction-error and discriminator-deviation anomaly signals via an OR-across-signals rule, and showed that this fusion is what gives the detector its adversarial robustness: a white-box adaptive attacker that fully defeats either signal in isolation is reduced to a 3.1% success rate against the fused score. Extending this evaluation to address final-review feedback, we found that a graph-theoretic encoder improves within-dataset detection accuracy but introduces a real, architecturally-grounded generalization weakness and an $8\times$ throughput cost, and that an initial measurement of catastrophic cross-dataset failure for both encoders was substantially, but not entirely, a unit-mismatch measurement artifact rather than a true model failure – a distinction that only became visible by investigating the result rather than reporting it at face value. We consider the identification and correction of that bug, and the subsequent isolation of the graph-theoretic encoder's genuine remaining weakness, to be as much a contribution of this work as the detector's original adversarial-robustness result, and we propose per-network recalibration as the concrete path toward closing the remaining generalization gap in a real deployment.

REFERENCES

[1] [Author(s)], "[GAN-based traffic augmentation for intrusion detection]," *[Venue]*, [Year]. *(Full citation to be completed from the original literature review bibliography.)*

[2] [Author(s)], "[Autoencoder-based anomaly scoring for network intrusion detection]," *[Venue]*, [Year]. *(Full citation to be completed.)*

[3] [Author(s)], "[VAE/latent-prior-matching network anomaly detector]," *[Venue]*, [Year]. *(Full citation to be completed.)*

[4] [Author(s)], "[Graph-based network traffic modeling with GCN/graph attention]," *[Venue]*, [Year]. *(Full citation to be completed.)*

[5] [Author(s)], "[Cross-domain/cross-dataset intrusion detection evaluation]," *[Venue]*, [Year]. *(Full citation to be completed.)*

[6] [Author(s)], "[Adaptive-adversary evaluation of ML-based network detectors]," *[Venue]*, [Year]. *(Full citation to be completed.)*

[7] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. 2nd IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009, pp. 1–6.